

Kubernetes and Cloud Native Associate - KCNA

Kubernetes Resources

Deployments

Welcome to this comprehensive guide on Kubernetes Deployments. In this article, we explain how Kubernetes Deployments streamline application rollouts, ensuring efficient updates and high availability in production environments.

When deploying an application like a web server, you typically run multiple instances to handle load and ensure uptime. As new versions of your application become available in the Docker registry, seamless upgrades are crucial. Since upgrading all instances simultaneously can disrupt active users, Kubernetes Deployments support rolling updates—updating instances one by one. Additionally, if an update introduces an error, you can quickly roll back changes. Deployments also allow you to bundle changes—such as updating the web server version, scaling resources, or adjusting resource allocations—and apply them together rather than individually.



Overview

Kubernetes Deployments build on foundational concepts: Pods encapsulate individual application instances, while ReplicaSets (or ReplicationControllers) manage multiple Pods. A Deployment is a higher-level construct that not only creates a ReplicaSet but also orchestrates rolling updates, rollbacks, and pause/resume operations.

Below is an example of a ReplicaSet definition used for context:

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-deployment
  labels:
    app: myapp
    type: front-end
spec:
  replicas: 3
  selector:
    matchLabels:
      type: front-end
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
        - name: nginx-container
          image: nginx
```

The corresponding Deployment definition is very similar, with the primary change being in the `kind` field. Here is the Deployment YAML:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
  labels:
    app: myapp
    type: front-end
spec:
  replicas: 3
  selector:
```

```
matchLabels:
  type: front-end
template:
  metadata:
    name: myapp-pod
    labels:
      app: myapp
      type: front-end
  spec:
    containers:
      - name: nginx-container
        image: nginx
```

To create the Deployment, save the above YAML content to a file (for example, `deployment-definition.yml`) and run:

```
kubectl create -f deployment-definition.yml
```

Once applied, you can verify the creation of your Deployment with the following commands:

1. List Deployments:

```
kubectl get deployments
```

2. Check the associated ReplicaSet:

```
kubectl get replicaset
```

3. View the Pods created by the Deployment:

```
kubectl get pods
```

For instance, you might see output similar to this:

```
> kubectl create -f deployment-definition.yml
deployment "myapp-deployment" created

> kubectl get deployments
NAME                                DESIRED    CURRENT    UP-TO-DATE    AVAILABLE    AGE
myapp-deployment                    3          3          3             3            21s

> kubectl get replicaset
NAME                                DESIRED    CURRENT    READY    AGE
myapp-deployment-6795844b58        3          3          3        2m

> kubectl get pods
NAME                                READY    STATUS    RESTARTS    AGE
myapp-deployment-6795844b58-5rbj1  1/1      Running   0           2m
myapp-deployment-6795844b58-h4w55  1/1      Running   0           2m
myapp-deployment-6795844b58-1fj hv  1/1      Running   0           2m
```

To see all created objects at once, use:

```
kubectl get all
```

This command displays your Deployment along with its associated ReplicaSet and Pods. For example:

```
> kubectl get all
NAME                                DESIRED    CURRENT    UP-TO-DATE    AVAILABLE    AGE
deploy/myapp-deployment            3          3          3             3            9h
NAME                                DESIRED    CURRENT    READY    AGE
rs/myapp-deployment-6795844b58     3          3          3        9h
NAME                                READY    STATUS    RESTARTS    AGE
po/myapp-deployment-6795844b58-5rbj1 1/1      Running   0           9h
po/myapp-deployment-6795844b58-h4w55 1/1      Running   0           9h
po/myapp-deployment-6795844b58-1fj hv 1/1      Running   0           9h
```



Next Steps

In upcoming sections, we will cover advanced Deployment features, including how to perform rolling updates, execute rollbacks, and pause/resume changes to manage your application lifecycle efficiently.

For more information, refer to the [Kubernetes Documentation](#) and explore other related resources such as [Kubernetes Basics](#).

Happy deploying!



Watch Video

Watch video content

Previous

◀ Demo ReplicaSets

Next

Demo Deployments ▶



Beta



● Search docs...

⌘K

Kubernetes and Cloud Native Associate - KCNA ▼